Lab 4: Vibe Coding Chat History

Repo #43: Real-Time Reaction Time Tester (SETAPESU26/43_reaction-time-tester)

Gnanasagar H | PES1UG24CS171 | Section 5 - 'C'

Tool used: Claude (Claude Code, model Claude Opus 5.5) inside VS Code. The assistant read the repository, ran the game, edited the code and recorded the videos. Below is the exchange, task by task, with the code it produced. Each prompt is the instruction for one step, written from the README task, so the job took one prompt for setup and one prompt per task.

Setup

Prompt

Clone https://github.com/SETAPESU26/43_reaction-time-tester, read the README, run the game and record a short video of the current behaviour before changing anything.

Claude

Cloned the repo and installed pygame 2.6.1. The game has main.py, game/game_engine.py and game/round.py. I ran it with a scripted player (one click while the screen is still grey, then normal clicks after green) and saved the run as before.mp4.

What the video shows: the early click is recorded as a normal reaction instead of a false start, every reaction includes the grey waiting time (the average comes out around 2000 ms instead of about 300 ms), and after round 5 the window just stays on the last screen while the results are printed to the console.

The cause is in Round.register_input(): it computes now - self.start_time (the round start) instead of now - self.go_time, and it never checks self.state, so a click in the waiting state is timed too.

Task 1: Refine input timing detection

Prompt

Fix Task 1. Reaction time must be measured from the moment the screen turns green, and a click during the grey wait must count as a false start, not a time. Don't let a false start use up a round.

Claude

register_input() now checks the state first. In the waiting state it switches the round to a new false_start state and returns None. In the go state it returns now - go_time. The engine only appends real times, counts false starts, shows a red "Too soon! False start" screen for 1.2 s and then replays the same round, so the round counter does not move.

```
    def register_input(self):
        # Returns the reaction time in ms, or None for a false start.
        # Timing is measured from the moment the screen turned green
        # (self.go_time), not from when the round started.
        now = pygame.time.get_ticks()
        if self.state == "waiting":
            self.state = "false_start"
            return None
        if self.state == "go":
            self.reaction_ms = now - self.go_time
```

```
            self.state = "result"
            return self.reaction_ms
        return None
```

Task 2: Game over condition

Prompt

Now Task 2. When all rounds are done, show a results screen in the window with every reaction time and the average, and wait for input instead of printing to the console.

Claude

Added _render_results(). Once game_over is set, render() draws a dark screen titled "Session complete!" with one line per round, the average and the number of false starts. The console print is gone. update() stops advancing rounds, so the screen stays until the player presses a key.

Task 3: Replay option

Prompt

Task 3. From the results screen let the player pick Easy, Medium or Hard to play again, with a different wait range and round count for each, or exit.

Claude

Added a DIFFICULTIES table and a start_session() method that resets the times, false starts and first round. On the results screen, keys 1, 2 and 3 start a new session, and Esc or Q sets quit_requested, which main.py checks to leave the loop. The current difficulty is shown at the bottom of the play screen.

Key	Difficulty	Rounds	Wait before green
1	Easy	3	1.0 to 2.0 s
2	Medium (default)	5	1.0 to 3.0 s
3	Hard	7	0.5 to 5.0 s

Task 4: Sound feedback

Prompt

Last one, Task 4. Add sounds for the go cue, a false start and the end of the session. Don't add audio files or new dependencies.

Claude

Added game/sounds.py, which builds three short tones in memory with pygame.mixer.Sound: a high beep on go, a low two-note buzz on a false start and a rising four-note chime when the session ends. If no audio device is available, mixer.init() raises pygame.error and the game carries on silently.

After all four tasks I recorded after.mp4 with the same scripted player. It shows a false start in round 2, reaction times of roughly 270 to 370 ms, the results screen, and a replay on Easy. The video has the sound track.

Files changed

File	Change
game/round.py	Timing from go_time; false_start state

game/game_engine.py	False-start handling, results screen, difficulty replay, sound hooks
game/sounds.py	New: generated go, false-start and session-end tones
main.py	Exits the loop when the player presses Esc on the results screen